What is Lazy Loading in Angular?

Lazy Loading is an Angular routing technique where **feature modules (or standalone feature routes)** are **loaded only when the user navigates to them**, instead of being loaded at application startup.

Why Lazy Loading is Important

In real-world Angular applications:

- The application grows large
- Initial bundle size increases
- First page load becomes slow

Lazy loading solves this by:

- Reducing initial bundle size
- Improving first load performance
- Loading features **on demand**

Eager Loading vs Lazy Loading

Aspect	**Eager Loading**	**Lazy Loading**
Load time	All modules loaded at startup	Loaded only when needed
Initial bundle	Large	Small
Performance	Slower initial load	Faster initial load
Best for	Small apps	Medium / Large apps

Where Lazy Loading is Used

Typical candidates for lazy loading:

- Admin module
- Reports module
- User management module

- Feature-heavy pages

Practical Example (Angular 20+)

We will build:

- **Home** → loaded eagerly
- **Admin** → loaded lazily

Step 1: Application Structure

src/

└── app/

  ├── app.routes.ts

  ├── app.component.ts

  ├── home/

  | └── home.component.ts

  └── admin/

    ├── admin.routes.ts

    └── admin.component.ts

Angular 20+ encourages **standalone components and route-based lazy loading**, not NgModules.

Step 2: Home Component (Eager Loaded)

```
// home.component.ts

import { Component } from '@angular/core';


@Component({

  standalone: true,
```

```
  selector: 'app-home',
  template: `<h2>Home Page</h2>`
})
export class HomeComponent {}
```

Step 3: Admin Component (Lazy Loaded Feature)

```
// admin.component.ts
import { Component } from '@angular/core';

@Component({
  standalone: true,
  selector: 'app-admin',
  template: `<h2>Admin Dashboard</h2>`
})
export class AdminComponent {}
```

Step 4: Admin Routes (Lazy Route Definition)

```
// admin.routes.ts
import { Routes } from '@angular/router';
import { AdminComponent } from './admin.component';

export const ADMIN_ROUTES: Routes = [
  {
    path: '',
    component: AdminComponent
  }
```

```
];
```

Step 5: App Routes with Lazy Loading

```
// app.routes.ts
import { Routes } from '@angular/router';
import { HomeComponent } from './home/home.component';


export const routes: Routes = [
 {
   path: '',
   component: HomeComponent
 },
 {
   path: 'admin',
   loadChildren: () =>
     import('./admin/admin.routes').then(m => m.ADMIN_ROUTES)
 }
];
```

Key Line (Lazy Loading Happens Here)

```
loadChildren: () => import('./admin/admin.routes')
```

- Angular creates a **separate JavaScript bundle**
- Bundle loads **only when /admin is visited**

Step 6: Navigation Example

```
<a routerLink="/">Home</a>
<a routerLink="/admin">Admin</a>
```

<router-outlet></router-outlet>

What Happens at Runtime?

1. App loads → **Home only**
2. Admin code is **not downloaded**
3. User clicks **Admin**
4. Angular fetches Admin bundle dynamically
5. Admin component renders

How to Verify Lazy Loading (Important for Students)

1. Open **Browser DevTools**
2. Go to **Network → JS**
3. Reload application
    - Only main bundle loads
4. Click **Admin**
    - New chunk file appears (lazy loaded)